

- If one target is given, ln creates a link to that file in the current directory.
- If the target directory (-t) option is given, or failing that if the last file is a directory and the no target directory (-T) option is given, ln creates a link to each target file in the specified directory, using the targets' names.

options: -b = backup, -d = directory, -f = force, -i = interactive, -L = logical, -n = no dereference, -P = physical, -r = relative, -s = symbolic, -S = suffix, -t directory = target directory, -T = no target directory, -v = verbose.

Module Three Assignment: File System Structure for a Multi-user environment.

1. Are devs equal or do they have a lead? Where does user identity live? Differences between User/service/system space? What should be most protected, isolated or scalable?
2. Permissions should enforce policy. Do the devs trust another? What groups should exist? Delete each other's work?
3. What fills the fastest? What happens at 100% fullness?

4. When should files survive if an original is deleted?
What's best for maintenance?

5. Will the environment survive if the company grows?

○ — ○ — ○ — ○ — ○ — ○ — ○ — ○ — ○ — ○ — ○ — ○

1. /home is a location for individual user docs, settings data. /srv has data served by the system, isolated from personal users. /opt is more for software it seems so they create lots of system folders. I'll presume the team are equal and are seeking only a shared dev workspace only. A new shared directory with subdirectories for the devs, I believe the standard directory implications are more confusing than creating something new.

Ex: /

└ home/

└ srv/

└ var/

└ etc/

└ opt/

└ Startup/

└ User data/ # Shared non personal data.

└ Projects/ # What devs are making,

└ logs/ # Project logs.

└ Configs/ # Project configs.

Cont. → /startup/
 userdata/
 <dataset>
 Projects/
 <projA> <projB>
 logs/
 <per-project>
 configs/
 <per-project>

I believe this setup will allow the team a shared directory without scattering project pieces into separate standard directories containing non-dev-team resources.

2. One dev-group, leader only deletion, everyone on the team can execute. I'll probably need a sticky bit with an ACL, though am unsure.

(leader, group, default behavior, deletion rules, etc.)

owner → leader, group → devs (what permissions?),

Special bits?

Devs should modify each others code, delete their own files, cannot delete other dev files.

Leader should delete anything, override permissions.

All should have equal access but not power to destroy; Shallow a working directory.

IF (775/777)
 non-dev users have too much access. 770, group is devs.

Cont. →

- /startup/ → rwx rwx --- ; devs can traverse/ manage the tree with outsiders lacking access
- /startup/projects → (770); Full collab for devs.
- Risk: Devs can overwrite code, without sticky bit devs can delete each others files. Mitigations: Sticky bit on shared project directories to prevent deleting other dev files, collab on git, set gid on project directories.
- Startup/configs and /startup/configs/<projA> → rwx rwx --- (770) Devs read/write, restricted to devs. Risk: Changes may not be tracked cleanly, if configs hold secrets data exposure is risked. Mitigations: Keep configs in git and enforce changes only via commit
- Startup/logs and /startup/logs/<projA> → (770), Risks: log tampering, disk exhaustion, sensitive data logged accidentally. Mitigations: logs treated as debug evidence, partition planning.
- Startup/userdata and /startup/data/<datasets> → (770) Risks: Dataset corruption, accidental deletion. Mitigation: Versioned datasets, perhaps a sticky bit.

3. One partition filling up could kill the whole system if logs explode, no isolation between system and dev workspace. With separate partitions, if /startup fills work stops but the OS remains. If /var fills logging stops/ services may fail, if /home fails users can't save files; if / fails then system instability, possible service failure, no temp files.

Cont. 7

With 1TB, /startup $\rightarrow$ 60%, 40% via LVM across /, var, home.

/Startup isolated as highest expected growth, if full development may stop but OS is stable. LVM allows dynamic resizing.

4. Symbolic Links.

1. Convenience path for devs: Devs need/want quick access to shared directories from home directly, without duplication. Symlink acts as a pointer to another path.

2. Standardizing tool locations: tools are installed in one place, but devs expect a common path. Common or expected paths pointed to tool locations so they can be found without copying scripts.

Hard Links

1. backup for single files: before changing files, a second name can be created as a rollback point.

2. Shared reference that must stay identical: A file needs to be in multiple locations while remaining identical